

Adopt Type Traits Variable Templates from Library Fundamentals TS for C++17

Table of Contents

- [Introduction](#)
- [Recommended for Immediate Adoption](#)
- [Proposed Wording](#)
- [Acknowledgements](#)
- [References](#)

Introduction

This paper recommends adopting the Type Trait Variable Templates from the Library Fundamentals TS for C++17. It does *not* propose removing them from the Library Fundamentals 2 TS, as parts of the TS rely on their presence, which is not available in the C++14 standard that is the base document for that series of TS.

Recommended for Immediate Adoption

Following the success of the `_t` alias templates for type traits, a matching set of `_v` variable templates have been proposed. The language support for variable templates arrived too late to confidently make this change for C++14, but experience since has shown that such variable templates are more succinct, can clean up the text in a similar way that the `_t` aliases have been widely adopted through the standard, and the author's experience using them in his own implementation of the standard type traits library is that code is much simpler when written using such variable templates directly, rather than turning a value into a type, then performing template manipulations on the type, before turning the type back into a value.

The impact on the standard is that many places that reference `some_trait<T>::value` would instead use `some_trait_v<T>`. The saving is not quite as great as in the case of alias templates, as there is no irksome typename to remove. However, the consistency of using `_t` and `_v` to refer to traits, and not using `::something` to extract meaning is compelling.

This paper proposes adding all of the `_v` alias templates from the Library Fundamentals working paper into the C++ Standard working paper, along with giving editorial direction to consistently use `_v` notation in wording throughout the standard. The paper author volunteers to help the editors with such a project. Adopting this feature now gives more time for the editors to catch any stray usage, before sending out the CD for ballot, hopefully next year.

It is important to bring this feature forward for adoption, even ahead of other Library Fundamentals components, as it has not only proven itself quickly, but the design is non-controversial, and the impact on how we document the standard library is significant. If we want to have a consistent use of the `_v` variables, rather than the more verbose `::value` traits throughout the standard, then we need to adopt this language as soon as possible, in order to perform the subsequent editorial cleanup in a reasonable timeframe, and so that new proposals can be written using the (expected) preferred language.

Proposed Wording

Add to the `<system_error>` synopsis, **19.5p2 [syserr]**:

```
// 19.5, System error support
template <class T> constexpr bool is_error_code_enum_v
    = is_error_code_enum<T>::value;
template <class T> constexpr bool is_error_condition_enum_v
    = is_error_condition_enum<T>::value;
```

Add to the `<tuple>` synopsis, **20.4.1p2 [tuple.general]**:

```
// 20.4.2.5, tuple helper classes
template <class T> constexpr size_t tuple_size_v
    = tuple_size<T>::value;
```

Add to the <memory> synopsis, **20.7.2p1 [memory.syn]**:

```
// 20.7.7, uses_allocator
template <class T, class Alloc> constexpr bool uses_allocator_v
    = uses_allocator<T, Alloc>::value;
```

Add to the <functional> synopsis, **20.9p2 [function.objects]**:

```
// 20.9.9, Function object binders
template <class T> constexpr bool is_bind_expression_v
    = is_bind_expression<T>::value;
template <class T> constexpr int is_placeholder_v
    = is_placeholder<T>::value;
```

Add to the <type_traits> synopsis, **20.10.2 [meta.type.synop]**:

```
// 20.10.4.1, primary type categories
template <class T> constexpr bool is_void_v
    = is_void<T>::value;
template <class T> constexpr bool is_null_pointer_v
    = is_null_pointer<T>::value;
template <class T> constexpr bool is_integral_v
    = is_integral<T>::value;
template <class T> constexpr bool is_floating_point_v
    = is_floating_point<T>::value;
template <class T> constexpr bool is_array_v
    = is_array<T>::value;
template <class T> constexpr bool is_pointer_v
    = is_pointer<T>::value;
template <class T> constexpr bool is_lvalue_reference_v
    = is_lvalue_reference<T>::value;
template <class T> constexpr bool is_rvalue_reference_v
    = is_rvalue_reference<T>::value;
template <class T> constexpr bool is_member_object_pointer_v
    = is_member_object_pointer<T>::value;
template <class T> constexpr bool is_member_function_pointer_v
    = is_member_function_pointer<T>::value;
template <class T> constexpr bool is_enum_v
    = is_enum<T>::value;
template <class T> constexpr bool is_union_v
    = is_union<T>::value;
template <class T> constexpr bool is_class_v
    = is_class<T>::value;
template <class T> constexpr bool is_function_v
    = is_function<T>::value;

// 20.10.4.2, composite type categories
template <class T> constexpr bool is_reference_v
    = is_reference<T>::value;
template <class T> constexpr bool is_arithmetic_v
    = is_arithmetic<T>::value;
template <class T> constexpr bool is_fundamental_v
    = is_fundamental<T>::value;
template <class T> constexpr bool is_object_v
    = is_object<T>::value;
template <class T> constexpr bool is_scalar_v
    = is_scalar<T>::value;
template <class T> constexpr bool is_compound_v
    = is_compound<T>::value;
template <class T> constexpr bool is_member_pointer_v
    = is_member_pointer<T>::value;

// 20.10.4.3, type properties
template <class T> constexpr bool is_const_v
    = is_const<T>::value;
template <class T> constexpr bool is_volatile_v
    = is_volatile<T>::value;
template <class T> constexpr bool is_trivial_v
    = is_trivial<T>::value;
template <class T> constexpr bool is_trivially_copyable_v
    = is_trivially_copyable<T>::value;
template <class T> constexpr bool is_standard_layout_v
    = is_standard_layout<T>::value;
template <class T> constexpr bool is_pod_v
    = is_pod<T>::value;
template <class T> constexpr bool is_literal_type_v
    = is_literal_type<T>::value;
template <class T> constexpr bool is_empty_v
```

```

    = is_empty<T>::value;
template <class T> constexpr bool is_polymorphic_v
    = is_polymorphic<T>::value;
template <class T> constexpr bool is_abstract_v
    = is_abstract<T>::value;
template <class T> constexpr bool is_final_v
    = is_final<T>::value;
template <class T> constexpr bool is_signed_v
    = is_signed<T>::value;
template <class T> constexpr bool is_unsigned_v
    = is_unsigned<T>::value;
template <class T, class... Args> constexpr bool is_constructible_v
    = is_constructible<T, Args...>::value;
template <class T> constexpr bool is_default_constructible_v
    = is_default_constructible<T>::value;
template <class T> constexpr bool is_copy_constructible_v
    = is_copy_constructible<T>::value;
template <class T> constexpr bool is_move_constructible_v
    = is_move_constructible<T>::value;
template <class T, class U> constexpr bool is_assignable_v
    = is_assignable<T, U>::value;
template <class T> constexpr bool is_copy_assignable_v
    = is_copy_assignable<T>::value;
template <class T> constexpr bool is_move_assignable_v
    = is_move_assignable<T>::value;
template <class T> constexpr bool is_destructible_v
    = is_destructible<T>::value;
template <class T, class... Args> constexpr bool is_trivially_constructible_v
    = is_trivially_constructible<T, Args...>::value;
template <class T> constexpr bool is_trivially_default_constructible_v
    = is_trivially_default_constructible<T>::value;
template <class T> constexpr bool is_trivially_copy_constructible_v
    = is_trivially_copy_constructible<T>::value;
template <class T> constexpr bool is_trivially_move_constructible_v
    = is_trivially_move_constructible<T>::value;
template <class T, class U> constexpr bool is_trivially_assignable_v
    = is_trivially_assignable<T, U>::value;
template <class T> constexpr bool is_trivially_copy_assignable_v
    = is_trivially_copy_assignable<T>::value;
template <class T> constexpr bool is_trivially_move_assignable_v
    = is_trivially_move_assignable<T>::value;
template <class T> constexpr bool is_trivially_destructible_v
    = is_trivially_destructible<T>::value;
template <class T, class... Args> constexpr bool is_nothrow_constructible_v
    = is_nothrow_constructible<T, Args...>::value;
template <class T> constexpr bool is_nothrow_default_constructible_v
    = is_nothrow_default_constructible<T>::value;
template <class T> constexpr bool is_nothrow_copy_constructible_v
    = is_nothrow_copy_constructible<T>::value;
template <class T> constexpr bool is_nothrow_move_constructible_v
    = is_nothrow_move_constructible<T>::value;
template <class T, class U> constexpr bool is_nothrow_assignable_v
    = is_nothrow_assignable<T, U>::value;
template <class T> constexpr bool is_nothrow_copy_assignable_v
    = is_nothrow_copy_assignable<T>::value;
template <class T> constexpr bool is_nothrow_move_assignable_v
    = is_nothrow_move_assignable<T>::value;
template <class T> constexpr bool is_nothrow_destructible_v
    = is_nothrow_destructible<T>::value;
template <class T> constexpr bool has_virtual_destructor_v
    = has_virtual_destructor<T>::value;

// See 20.10.5, type property queries
template <class T> constexpr size_t alignment_of_v
    = alignment_of<T>::value;
template <class T> constexpr size_t rank_v
    = rank<T>::value;
template <class T, unsigned I = 0> constexpr size_t extent_v
    = extent<T, I>::value;

// See 20.10.6, type relations
template <class T, class U> constexpr bool is_same_v
    = is_same<T, U>::value;
template <class Base, class Derived> constexpr bool is_base_of_v
    = is_base_of<Base, Derived>::value;
template <class From, class To> constexpr bool is_convertible_v
    = is_convertible<From, To>::value;

```

Add to the <ratio> synopsis, **20.11.2 [ratio.syn]**:

```
// 20.11.5, ratio comparison
template <class R1, class R2> constexpr bool ratio_equal_v
    = ratio_equal<R1, R2>::value;
template <class R1, class R2> constexpr bool ratio_not_equal_v
    = ratio_not_equal<R1, R2>::value;
template <class R1, class R2> constexpr bool ratio_less_v
    = ratio_less<R1, R2>::value;
template <class R1, class R2> constexpr bool ratio_less_equal_v
    = ratio_less_equal<R1, R2>::value;
template <class R1, class R2> constexpr bool ratio_greater_v
    = ratio_greater<R1, R2>::value;
template <class R1, class R2> constexpr bool ratio_greater_equal_v
    = ratio_greater_equal<R1, R2>::value;
```

Add to the <chrono> synopsis, **20.12.2 [time.syn]**:

```
// 20.12.4, customization traits
template <class Rep> constexpr bool treat_as_floating_point_v
    = treat_as_floating_point<Rep>::value;
```

Acknowledgements

Thanks to Walter Brown for proposing and persevering with the variable template language feature for C++14, and to Stephan T. Lavavej for proposing their idiomatic use in the type traits library, and beyond.

References

[N3932](#) Variable Templates For Type Traits, Stephan T. Lavavej

[N4480](#) Library Fundamentals TS (Final Draft)